



REPUBLIC OF YEMEN
SANA'A UNIVERSITY
FACULTY OF ENGINEERING
MECHATRONICS ENGINEERING DEPARTMENT



DC Motor Speed Control Using Embedded System

Done by:

Hamzah Alkhalaf 202274127

Suhib Almalhani 202174006

Musa ALsalmani 202174202

Hasan Moqbel 202274198

System (Parallel)

Supervised by:

Eng. Mohammed Al-Zaghir

DC Motor Speed Control Using Embedded System.....	1
1. Introduction	3
2. System Overview	3
Components	3
Functional Requirements.....	3
3. System Design	4
a. Motor Control Using L298N and PWM	4
b. Speed Feedback Using Rotary Encoder	4
c. PID Controller for Speed Control	4
d. Wireless Communication with ESP32 (Optional)	4
e. Real-Time Monitoring Using I2C LCD	5
4. Implementation.....	5
a. Hardware Configuration	5
b. Software Design	5
c. Simulation and Testing	5
5. Results and Analysis	6
a. Speed Control Performance	6
b. Wireless Communication	6
c. Real-Time Display	6
d. System Metrics.....	6
6. Conclusion.....	6

1. Introduction

This report details the design and implementation of a DC motor speed control system using an **ATmega328P microcontroller**, **L298N motor driver**, and a **rotary encoder** for speed feedback. An **I2C-based LCD** displays real-time speed and system parameters. Additionally, an optional **ESP32 module** provides wireless communication via **Wi-Fi** or **Bluetooth**. The system leverages key features of embedded systems, such as digital and analog interfaces, PWM, interrupts, timers, UART, and I2C.

2. System Overview

Components

1. **ATmega328P Microcontroller**
 - Core controller for the system.
 - Key features:
 - **Digital I/O:** For motor control and encoder feedback.
 - **Analog Inputs:** For optional sensors.
 - **PWM:** For motor speed control.
 - **Interrupts:** For real-time speed measurement.
 - **Timers:** For precise timing of control loops.
 - **UART:** For serial communication with the ESP32.
 - **I2C Interface:** For connecting the LCD.
2. **L298N Motor Driver**
 - Dual H-bridge driver for controlling the DC motor's speed and direction.
 - Controlled via PWM and digital I/O signals.
3. **Rotary Encoder**
 - Provides speed feedback by generating pulses corresponding to motor rotation.
 - The pulses are processed using interrupts for accurate speed measurement.
4. **ESP32 (Optional)**
 - Adds Wi-Fi or Bluetooth connectivity for remote control and monitoring.
5. **I2C LCD Display**
 - Displays system data such as motor speed, setpoint, and error.

Functional Requirements

- **Digital Inputs/Outputs:** For motor control and encoder signal processing.

- **Analog Inputs:** For additional sensors, e.g., temperature monitoring.
- **PWM:** To control motor speed via the L298N driver.
- **Interrupts:** For capturing encoder pulses in real-time.
- **Timers:** For scheduling the PID control algorithm and ensuring consistent sampling.
- **UART Interface:** For communication with ESP32 or debugging.
- **I2C Interface:** To drive the LCD display.
- **Wi-Fi/Bluetooth:** For remote operation (optional).

3. System Design

a. Motor Control Using L298N and PWM

The ATmega328P generates PWM signals to control the motor speed. The L298N motor driver amplifies these signals and drives the motor. Direction control is achieved using digital outputs.

b. Speed Feedback Using Rotary Encoder

The rotary encoder provides real-time feedback on motor speed by generating pulses proportional to the motor's rotation. These pulses are captured using external interrupts on the ATmega328P.

c. PID Controller for Speed Control

A Proportional-Integral-Derivative (PID) controller is implemented to minimize speed error:

$$u(t) = K_p e(t) + K_i \int e(t) dt + K_d \frac{de(t)}{dt}$$

Where:

- $e(t)$ = Speed error (setpoint - measured speed).
- K_p , K_i and K_d = Proportional, Integral, and Derivative gains.

The PID controller adjusts the PWM duty cycle to maintain the desired speed.

d. Wireless Communication with ESP32 (Optional)

The ESP32 enables remote control and monitoring via Wi-Fi or Bluetooth. It communicates with the ATmega328P using UART. Users can monitor speed and system parameters or send control commands through a smartphone app or web interface.

e. Real-Time Monitoring Using I2C LCD

An I2C LCD displays:

- Current motor speed
- Setpoint speed
- Error value
- System status

4. Implementation

a. Hardware Configuration

1. **ATmega328P Microcontroller:**
 - PWM output connected to L298N for speed control.
 - Digital I/O for motor direction control and encoder feedback.
 - UART for communication with ESP32.
 - I2C interface for LCD communication.
2. **L298N Motor Driver:**
 - Controls motor speed and direction using PWM and digital signals.
3. **Rotary Encoder:**
 - Encoder outputs connected to ATmega328P external interrupt pins for pulse counting.
4. **ESP32 Module (Optional):**
 - UART connection to ATmega328P for wireless communication.
5. **I2C LCD Display:**
 - Displays real-time data via the I2C interface.

b. Software Design

1. **Interrupt Service Routine (ISR) for Encoder:**
 - Captures encoder pulses to calculate motor speed.
2. **PWM Control:**
 - Timer1 configured to generate PWM signals for motor control.
3. **PID Algorithm:**
 - Executes periodically using Timer interrupts to adjust PWM duty cycle based on speed error.
4. **I2C Communication:**
 - Updates the LCD with real-time data.
5. **UART Communication:**
 - Exchanges data with the ESP32 for remote monitoring and control.

c. Simulation and Testing

The system was simulated in Proteus to verify hardware and software functionality. After successful simulation, the design was implemented and tested on actual hardware.

5. Results and Analysis

a. Speed Control Performance

The system demonstrated stable speed control with minimal steady-state error. The PID controller effectively responded to changes in load and setpoint, maintaining desired speed.

b. Wireless Communication

With the ESP32 module, the system successfully transmitted data and received control commands via Wi-Fi and Bluetooth.

c. Real-Time Display

The I2C LCD provided accurate, real-time feedback, enhancing system usability.

d. System Metrics

- **Settling time:** Minimal, depending on PID tuning.
- **Overshoot:** Reduced with appropriate PID parameters.
- **Steady-state error:** Negligible after tuning.

6. Conclusion

The DC motor speed control system, based on the ATmega328P microcontroller, effectively leverages embedded system features for precise and reliable control. The system's modular design supports wireless connectivity and real-time monitoring, making it suitable for various automation applications.